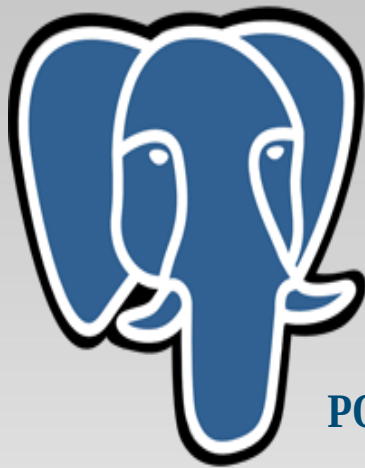




PostgreSQL

POSTGRES DATABASE ADMINISTRATION



STEPHEN NJOROGHE

LinkedIn: www.linkedin.com/in/stephen-njoroge

[SENIOR DATABASE ADMINISTRATOR|ORACLE| POSTGRESQL | MSSQL|OCI|AWS]

Chapter 5: PostgreSQL Backup and Recovery

About the Author:

I am **Stephen Njoroge**, a Senior Database Engineer with over 8+ years of expertise in designing, securing and optimizing high-performance database systems across the banking and fintech sectors. My core technical proficiency spans major databases: Oracle, PostgreSQL, MS SQL Server and MySQL-along with advanced scripting in SQL and Python for automation and data pipeline development. I architect and manage cloud and virtualization solutions on OCI, AWS, and Proxmox for onprem systems and implement comprehensive DevOps practices using monitoring tools like Prometheus and Grafana and IT service management via iTop. Further, I deliver advanced data analytics and visualization using Tableau, Power BI and Oracle BI. As a Certified Data Protection Officer, I integrate robust data protection, compliance and security measures including vulnerability assessment and role-based access control into every layer of the data infrastructure.

System Environment

- OS: Ubuntu 24.04.3
- PostgreSQL Version: 18.1
- Location: `/var/lib/postgresql/18/main/`

Introduction

In this chapter, we learn about the various backup and recovery methods essential to maintaining data integrity and availability. PostgreSQL offers a wide range of tools to perform backup and restore operations based on your requirements and data loss Scenario.

We learn the following topics in this chapter:

- Type of backups in PostgreSQL
- SQL Dump
- File System-Level backup
- Continuous archiving and point-in-time recovery (PITR)
- Using external tools – Barman and pgBackRest

SQL Dump (`pg_dump` and `pg_restore`)

▪ Setup and Prerequisites

```
# create a HRM database with some data for this demo
psql -U postgres -d hrmdb

-- Create training tables
CREATE TABLE employees (
    employee_id SERIAL PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50),
    email VARCHAR(100),
    hire_date DATE,
    salary NUMERIC(10,2),
    department_id INTEGER,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE departments (
    department_id SERIAL PRIMARY KEY,
    department_name VARCHAR(100),
    manager_id INTEGER,
    location VARCHAR(100),
    budget NUMERIC(15,2)
);

CREATE TABLE salaries (
    salary_id SERIAL PRIMARY KEY,
    employee_id INTEGER REFERENCES employees(employee_id),
    salary_amount NUMERIC(10,2),
    effective_date DATE,
    end_date DATE
```

```

);

CREATE TABLE audit_logs (
    log_id SERIAL PRIMARY KEY,
    table_name VARCHAR(50),
    action VARCHAR(20),
    changed_by VARCHAR(50),
    change_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    old_data JSONB,
    new_data JSONB
);

-- Insert sample data
INSERT INTO departments (department_name, manager_id, location, budget) VALUES
    ('Engineering', 1, 'New York', 1000000),
    ('Human Resources', 2, 'Chicago', 500000),
    ('Sales', 3, 'Los Angeles', 750000),
    ('Marketing', 4, 'New York', 600000),
    ('Finance', 5, 'Chicago', 800000);

INSERT INTO employees (first_name, last_name, email, hire_date, salary, department_id) VALUES
    ('John', 'Doe', 'john.doe@company.com', '2020-01-15', 85000, 1),
    ('Jane', 'Smith', 'jane.smith@company.com', '2019-03-20', 95000, 1),
    ('Bob', 'Johnson', 'bob.johnson@company.com', '2021-06-10', 65000, 2),
    ('Alice', 'Williams', 'alice.williams@company.com', '2018-11-05', 110000, 3),
    ('Charlie', 'Brown', 'charlie.brown@company.com', '2022-01-10', 55000, 2),
    ('Diana', 'Prince', 'diana.prince@company.com', '2020-09-15', 105000, 3),
    ('Edward', 'Norton', 'edward.norton@company.com', '2019-07-22', 78000, 4),
    ('Fiona', 'Apple', 'fiona.apple@company.com', '2021-03-01', 72000, 4),
    ('George', 'Clooney', 'george.clooney@company.com', '2018-05-12', 120000, 5),
    ('Hannah', 'Montana', 'hannah.montana@company.com', '2022-08-20', 58000, 5);

INSERT INTO salaries (employee_id, salary_amount, effective_date, end_date) VALUES
    (1, 80000, '2020-01-15', '2021-01-14'),
    (1, 85000, '2021-01-15', NULL),
    (2, 90000, '2019-03-20', '2020-03-19'),
    (2, 95000, '2020-03-20', NULL),
    (3, 65000, '2021-06-10', NULL);

-- Create indexes
CREATE INDEX idx_employees_email ON employees(email);
CREATE INDEX idx_employees_department ON employees(department_id);
CREATE INDEX idx_salaries_employee ON salaries(employee_id);
CREATE INDEX idx_audit_logs_date ON audit_logs(change_date);

-- Create a view
CREATE VIEW employee_summary AS
SELECT
    e.employee_id,
    e.first_name,
    e.last_name,
    e.email,
    d.department_name,
    e.salary,
    e.hire_date
FROM employees e
JOIN departments d ON e.department_id = d.department_id;

```

```

-- Create a function
CREATE OR REPLACE FUNCTION get_employees_by_department(dept_id INTEGER)
RETURNS TABLE (
    employee_id INTEGER,
    full_name TEXT,
    email VARCHAR,
    salary NUMERIC
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        e.employee_id,
        e.first_name || ' ' || e.last_name,
        e.email,
        e.salary
    FROM employees e
    WHERE e.department_id = dept_id
    ORDER BY e.last_name;
END;
$$ LANGUAGE plpgsql;

```

```

hrmdb=# -- Create a function
CREATE OR REPLACE FUNCTION get_employees_by_department(dept_id INTEGER)
RETURNS TABLE (
    employee_id INTEGER,
    full_name TEXT,
    email VARCHAR,
    salary NUMERIC
) AS $$
BEGIN
    RETURN QUERY
    SELECT
        e.employee_id,
        e.first_name || ' ' || e.last_name,
        e.email,
        e.salary
    FROM employees e
    WHERE e.department_id = dept_id
    ORDER BY e.last_name;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
hrmdb=#

```

```

-- Create a trigger for audit logging
CREATE OR REPLACE FUNCTION audit_employee_changes()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO audit_logs (table_name, action, changed_by, old_data, new_data)
    VALUES (
        'employees',
        TG_OP,
        current_user,
        CASE WHEN TG_OP = 'DELETE' THEN row_to_json(OLD) ELSE NULL END,
        CASE WHEN TG_OP IN ('INSERT', 'UPDATE') THEN row_to_json(NEW) ELSE NULL END
    );
    RETURN NEW;
END;

```

```
$$ LANGUAGE plpgsql;
```

```
postgres=# \c hrmdb
You are now connected to database "hrmdb" as user "postgres".
hrmdb=# CREATE OR REPLACE FUNCTION audit_employee_changes()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO audit_logs (table_name, action, changed_by, old_data, new_data)
    VALUES (
        'employees',
        TG_OP,
        current_user,
        CASE WHEN TG_OP = 'DELETE' THEN row_to_json(OLD) ELSE NULL END,
        CASE WHEN TG_OP IN ('INSERT', 'UPDATE') THEN row_to_json(NEW) ELSE NULL END
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE FUNCTION
hrmdb=# |
```

```
CREATE TRIGGER employee_audit_trigger
AFTER INSERT OR UPDATE OR DELETE ON employees
FOR EACH ROW EXECUTE FUNCTION audit_employee_changes();
```

```
hrmdb=# CREATE TRIGGER employee_audit_trigger
AFTER INSERT OR UPDATE OR DELETE ON employees
FOR EACH ROW EXECUTE FUNCTION audit_employee_changes();
CREATE TRIGGER
hrmdb=#
```

```
-- Grant privileges
GRANT SELECT ON employee_summary TO public;
GRANT EXECUTE ON FUNCTION get_employees_by_department(INTEGER) TO public;
```

```
--crosscheck the database objects
-- tables
```

```
hrmdb=# \dt
          List of tables
Schema |   Name   | Type | Owner
-----+-----+-----+-----
public | audit_logs | table | postgres
public | departments | table | postgres
public | employees | table | postgres
public | salaries | table | postgres
(4 rows)

hrmdb=#
```

```
hrmdb=# \dv
```

List of views			
Schema	Name	Type	Owner
public	employee_summary	view	postgres

(1 row)

```
hrmdb=# \di
```

List of indexes				
Schema	Name	Type	Owner	Table
public	audit_logs_pkey	index	postgres	audit_logs
public	departments_pkey	index	postgres	departments
public	employees_pkey	index	postgres	employees
public	idx_audit_logs_date	index	postgres	audit_logs
public	idx_employees_department	index	postgres	employees
public	idx_employees_email	index	postgres	employees
public	idx_salaries_employee	index	postgres	salaries
public	salaries_pkey	index	postgres	salaries

(8 rows)

Backup And Restore Scenarios.

Full Database Backup and Restore

- Database Dump (Custom Format)

```
# Create backup directory
```

```
mkdir -p /hrmdb_backups/scenario1
```

```
echo "=== SCENARIO 1: Full Database Backup ==="
```

```
# 1. Check database size before backup
```

```
echo "Database size before backup:"
```

```
psql -U postgres -d hrmdb -c "SELECT pg_size_pretty(pg_database_size('hrmdb')) as db_size;"
```

```
postgres@998-986-02:/home/steve$ echo "Database size before backup:"
psql -U postgres -d hrmdb -c "SELECT pg_size_pretty(pg_database_size('hrmdb')) as db_size;"
Database size before backup:
 db_size
-----
 8022 kB
(1 row)

postgres@998-986-02:/home/steve$ |
```

```
# 2. Get row counts before backup
```

```
echo -e "\nRow counts before backup:"
```

```
psql -U postgres -d hrmdb -c "SELECT 'employees' as table_name, COUNT(*) as row_count FROM employees
UNION ALL SELECT 'departments', COUNT(*) FROM departments UNION ALL SELECT 'salaries', COUNT(*) FROM
salaries UNION ALL SELECT 'audit_logs', COUNT(*) FROM audit_logs;"
```

```
postgres@998-986-02:/home/steve$ echo -e "\nRow counts before backup:"
psql -U postgres -d hrmdb -c "SELECT 'employees' as table_name, COUNT(*) as row_count FROM employees UNION ALL SELECT 'departments',
COUNT(*) FROM departments UNION ALL SELECT 'salaries', COUNT(*) FROM salaries UNION ALL SELECT 'audit_logs', COUNT(*) FROM audit_logs
;"

Row counts before backup:
 table_name | row_count
-----+-----
 employees  |         10
 departments |          5
 salaries   |          5
 audit_logs  |          0
(4 rows)

postgres@998-986-02:/home/steve$ |
```

3. Perform full backup in custom format

```
BACKUP_FILE="/hrmdb_backups/scenario1/hrmdb_full_$(date +%Y%m%d_%H%M%S).dump"
echo -e "\nPerforming full backup to: $BACKUP_FILE"
pg_dump -U postgres -h localhost -Fc -v hrmdb > "$BACKUP_FILE"
```

```
postgres@998-986-02:/home/steve$ BACKUP_FILE="/hrmdb_backups/scenario1/hrmdb_full_$(date +%Y%m%d_%H%M%S).dump"
echo -e "\nPerforming full backup to: $BACKUP_FILE"
pg_dump -U postgres -h localhost -Fc -v hrmdb > "$BACKUP_FILE"
```

```
Performing full backup to: /hrmdb_backups/scenario1/hrmdb_full_20260214_105833.dump
Password:
pg_dump: executing SELECT pg_catalog.set_config('search_path', '', false);
pg_dump: last built-in OID is 16383
pg_dump: reading extensions
pg_dump: identifying extension members
pg_dump: reading schemas
pg_dump: reading user-defined tables
pg_dump: reading user-defined functions
pg_dump: reading user-defined types
pg_dump: reading procedural languages
pg_dump: reading user-defined aggregate functions
pg_dump: reading user-defined operators
pg_dump: reading user-defined access methods
pg_dump: reading user-defined operator classes
pg_dump: reading user-defined operator families
pg_dump: reading user-defined text search parsers
pg_dump: reading user-defined text search templates
pg_dump: reading user-defined text search dictionaries
pg_dump: reading user-defined text search configurations
pg_dump: reading user-defined foreign-data wrappers
pg_dump: reading user-defined foreign servers
pg_dump: reading default privileges
```

4. Verify backup file

```
echo -e "\nBackup file details:"
ls -lh "$BACKUP_FILE"
```

```
postgres@998-986-02:/home/steve$ echo -e "\nBackup file details:"
ls -lh "$BACKUP_FILE"
```

```
Backup file details:
-rw-rw-r-- 1 postgres postgres 28K Feb 14 10:58 /hrmdb_backups/scenario1/hrmdb_full_20260214_105833.dump
postgres@998-986-02:/home/steve$
```

5. List contents of backup

```
echo -e "\nFirst 10 items in backup:"
pg_restore -1 "$BACKUP_FILE" | head -10
```

```

postgres@998-986-02:/home/steve$ echo -e "\nFirst 10 items in backup:"
pg_restore -l "$BACKUP_FILE" | head -10

First 10 items in backup:
;
; Archive created at 2026-02-14 10:58:33 MSK
; dbname: hrmdb
; TOC Entries: 42
; Compression: gzip
; Dump Version: 1.16-0
; Format: CUSTOM
; Integer: 4 bytes
; Offset: 8 bytes
; Dumped from database version: 18.1 (Ubuntu 18.1-1.pgdg24.04+2)
postgres@998-986-02:/home/steve$

```

■ Full Database Restore

```
echo "=== SCENARIO 1: Full Database Restore ==="
```

```
# 1. Create a new database for restore
```

```
createdb -U postgres hrmdb_restored_full
```

```
# 2. Restore the full backup
```

```
echo "Restoring full backup to hrmdb_restored_full..."
```

```
pg_restore -U postgres -d hrmdb_restored_full -v "$BACKUP_FILE"
```

```

postgres@998-986-02:/home/steve$ pg_restore -U postgres -d hrmdb_restored_full -v "$BACKUP_FILE"
pg_restore: connecting to database for restore
pg_restore: executing SELECT pg_catalog.set_config('search_path', '', false);
pg_restore: creating FUNCTION "public.audit_employee_changes()"
pg_restore: creating FUNCTION "public.get_employees_by_department(integer)"
pg_restore: creating TABLE "public.audit_logs"
pg_restore: creating SEQUENCE "public.audit_logs_log_id_seq"
pg_restore: creating SEQUENCE OWNED BY "public.audit_logs_log_id_seq"
pg_restore: creating TABLE "public.departments"
pg_restore: creating SEQUENCE "public.departments_department_id_seq"
pg_restore: creating SEQUENCE OWNED BY "public.departments_department_id_seq"
pg_restore: creating TABLE "public.employees"
pg_restore: creating VIEW "public.employee_summary"
pg_restore: creating SEQUENCE "public.employees_employee_id_seq"
pg_restore: creating SEQUENCE OWNED BY "public.employees_employee_id_seq"
pg_restore: creating TABLE "public.salaries"
pg_restore: creating SEQUENCE "public.salaries_salary_id_seq"
pg_restore: creating SEQUENCE OWNED BY "public.salaries_salary_id_seq"

```

```
# 3. Verify restore
```

```
echo -e "\nVerifying restore:"
```

```
psql -U postgres -d hrmdb_restored_full -c "SELECT COUNT(*) as employee_count FROM employees;"
```

```
psql -U postgres -d hrmdb_restored_full -c "SELECT COUNT(*) as department_count FROM departments;"
```

```
postgres@998-986-02:/home/steve$ psql -U postgres -d hrmdb_restored_full -c "SELECT COUNT(*) as employee_count FROM employees;"
employee_count
-----
10
(1 row)

postgres@998-986-02:/home/steve$ psql -U postgres -d hrmdb_restored_full -c "SELECT COUNT(*) as department_count FROM departments;"
department_count
-----
5
(1 row)

postgres@998-986-02:/home/steve$ |
```

4. Check if views and functions were restored

echo -e "\nChecking views and functions:"

psql -U postgres -d hrmdb_restored_full -c "\dv"

```
postgres@998-986-02:/home/steve$ psql -U postgres -d hrmdb_restored_full -c "\dv"
psql -U postgres -d hrmdb_restored_full -c "\df"

List of views
Schema | Name | Type | Owner
-----+-----+-----+-----
public | employee_summary | view | postgres
(1 row)
```

```
postgres@998-986-02:/home/steve$ psql -U postgres -d hrmdb_restored_full -c "\dv"
List of views
Schema | Name | Type | Owner
-----+-----+-----+-----
public | employee_summary | view | postgres
(1 row)

postgres@998-986-02:/home/steve$
```

psql -U postgres -d hrmdb_restored_full -c "\df"

```
Schema | Name | Result data type | Argumen
-----+-----+-----+-----
public | audit_employee_changes | trigger |
public | get_employees_by_department | TABLE(employee_id integer, full_name text, email character varying, salary numeric) | dept_id integer
(2 rows)
```

Selective Backup and Restore (Specific Tables)

Backup Only Specific Tables

```
mkdir -p /hrmdb_backups/scenario2
```

```
echo "=== SCENARIO 2: Selective Table Backup ==="
```

1. Backup only employee-related tables

```
BACKUP_EMPLOYEES="/hrmdb_backups/scenario2/hrmdb_employees_only_$(date +%Y%m%d).dump"
```

```
echo "Backing up only employees and salaries tables..."
```

```
pg_dump -U postgres -Fc \
  -t employees \
  -t salaries \
  -t departments \
  hrmdb > "$BACKUP_EMPLOYEES"
```

```

postgres@998-986-02:/home/steve$ BACKUP_EMPLOYEES="/hrmdb_backups/scenario2/hrmdb_employees_only_$(date +%Y%m%d).dump"
echo "Backing up only employees and salaries tables..."
pg_dump -U postgres -Fc \
    -t employees \
    -t salaries \
    -t departments \
    hrmdb > "$BACKUP_EMPLOYEES"
Backing up only employees and salaries tables...
postgres@998-986-02:/home/steve$ |

```

2. Verify backup contents

```

echo -e "\nContents of selective backup:"
pg_restore -l "$BACKUP_EMPLOYEES" | grep -E "TABLE (employees|salaries|departments)"

```

```

postgres@998-986-02:/hrmdb_backups/scenario2$ # Check what's actually in the backup
pg_restore -l /hrmdb_backups/scenario2/hrmdb_employees_only_20260214.dump | grep -E "TABLE.*(employees|salaries|departments)"
222; 1259 51537 TABLE public departments postgres
220; 1259 51528 TABLE public employees postgres
224; 1259 51545 TABLE public salaries postgres
3451; 0 51537 TABLE DATA public departments postgres
3449; 0 51528 TABLE DATA public employees postgres
3453; 0 51545 TABLE DATA public salaries postgres
postgres@998-986-02:/hrmdb_backups/scenario2$

```

3. Backup excluding audit logs

```

BACKUP_NO_AUDIT="/hrmdb_backups/scenario2/hrmdb_no_audit_$(date +%Y%m%d).dump"
echo -e "\nBacking up entire database except audit_logs..."
pg_dump -U postgres -Fc \
    -T audit_logs \
    hrmdb > "$BACKUP_NO_AUDIT"

```

```

postgres@998-986-02:/hrmdb_backups/scenario2$ BACKUP_NO_AUDIT="/hrmdb_backups/scenario2/hrmdb_no_audit_$(date +%Y%m%d).dump"
echo -e "\nBacking up entire database except audit_logs..."
pg_dump -U postgres -Fc \
    -T audit_logs \
    hrmdb > "$BACKUP_NO_AUDIT"

Backing up entire database except audit_logs...
postgres@998-986-02:/hrmdb_backups/scenario2$ ls -lrt
total 44
-rw-rw-r-- 1 postgres postgres 18648 Feb 14 16:06 hrmdb_employees_only_20260214.dump
-rw-rw-r-- 1 postgres postgres 23808 Feb 14 16:10 hrmdb_no_audit_20260214.dump
postgres@998-986-02:/hrmdb_backups/scenario2$ |

```

4. Compare sizes

```

echo -e "\nBackup sizes comparison:"
ls -lh "$BACKUP_EMPLOYEES" "$BACKUP_NO_AUDIT"

```

```

postgres@998-986-02:/hrmdb_backups/scenario2$ echo -e "\nBackup sizes comparison:"
ls -lh "$BACKUP_EMPLOYEES" "$BACKUP_NO_AUDIT"

Backup sizes comparison:
-rw-rw-r-- 1 postgres postgres 19K Feb 14 16:06 /hrmdb_backups/scenario2/hrmdb_employees_only_20260214.dump
-rw-rw-r-- 1 postgres postgres 24K Feb 14 16:10 /hrmdb_backups/scenario2/hrmdb_no_audit_20260214.dump
postgres@998-986-02:/hrmdb_backups/scenario2$ |

```

▪ Restore Specific Tables

```
echo "=== SCENARIO 2: Selective Table Restore ==="
```

```
# 1. Create a test database
```

```
createdb -U postgres hrmdb_selective_test
```

```
# 2. Restore only specific tables
```

```
echo "Restoring only employees and departments tables..."
```

```
pg_restore -U postgres -d hrmdb_selective_test \  
-t employees \  
-t departments \  
"$BACKUP_EMPLOYEES"
```

```
postgres@998-986-02:~$ echo "Restoring only employees and departments tables..."  
pg_restore -U postgres -d hrmdb_selective_test \  
-t employees \  
-t departments \  
"$BACKUP_EMPLOYEES"  
Restoring only employees and departments tables...  
postgres@998-986-02:~$ |
```

```
# 3. Verify only selected tables exist
```

```
echo -e "\nTables in restored database:"
```

```
psql -U postgres -d hrmdb_selective_test -c "\dt"
```

```
postgres@998-986-02:~$ echo -e "\nTables in restored database:"  
psql -U postgres -d hrmdb_selective_test -c "\dt"  
  
Tables in restored database:  
List of tables  
Schema | Name | Type | Owner  
-----+-----+-----+-----  
public | departments | table | postgres  
public | employees | table | postgres  
(2 rows)  
  
postgres@998-986-02:~$
```

```
# 4. Try to restore a single table from full backup
```

```
pg_restore -U postgres -d hrmdb_selective_test \  
-t salaries \  
--clean --if-exists \  
"$BACKUP_FILE"
```

```
# 5. Verify all three tables now exist
```

```
echo -e "\nFinal table list:"
```

```
psql -U postgres -d hrmdb_selective_test -c "\dt"
```

```
postgres@998-986-02:~$ echo -e "\nFinal table list:"
psql -U postgres -d hrmdb_selective_test -c "\dt"
```

Final table list:

List of tables			
Schema	Name	Type	Owner
public	departments	table	postgres
public	employees	table	postgres
public	salaries	table	postgres

(3 rows)

```
postgres@998-986-02:~$
```

Schema-Only and Data-Only Operations

■ Schema-Only and Data-Only Backups

```
mkdir -p /hrmdb_backups/scenario3
```

```
echo "=== SCENARIO 3: Schema and Data Separation ==="
```

1. Schema-only backup

```
SCHEMA_BACKUP="/hrmdb_backups/scenario3/hrmdb_schema_only_$(date +%Y%m%d).sql"
```

```
echo "Creating schema-only backup..."
```

```
pg_dump -U postgres -s hrmdb > "$SCHEMA_BACKUP"
```

2. Data-only backup

```
DATA_BACKUP="/hrmdb_backups/scenario3/hrmdb_data_only_$(date +%Y%m%d).sql"
```

```
echo "Creating data-only backup..."
```

```
pg_dump -U postgres -a hrmdb > "$DATA_BACKUP"
```

3. Compare file sizes

```
echo -e "\nBackup sizes:"
```

```
ls -lh "$SCHEMA_BACKUP" "$DATA_BACKUP"
```

```
postgres@998-986-02:~$ # 1. Schema-only backup
```

```
SCHEMA_BACKUP="/hrmdb_backups/scenario3/hrmdb_schema_only_$(date +%Y%m%d).sql"
```

```
echo "Creating schema-only backup..."
```

```
pg_dump -U postgres -s hrmdb > "$SCHEMA_BACKUP"
```

```
Creating schema-only backup...
```

```
postgres@998-986-02:~$ DATA_BACKUP="/hrmdb_backups/scenario3/hrmdb_data_only_$(date +%Y%m%d).sql"
```

```
echo "Creating data-only backup..."
```

```
pg_dump -U postgres -a hrmdb > "$DATA_BACKUP"
```

```
Creating data-only backup...
```

```
postgres@998-986-02:~$ echo -e "\nBackup sizes:"
```

```
ls -lh "$SCHEMA_BACKUP" "$DATA_BACKUP"
```

```
Backup sizes:
```

```
-rw-rw-r-- 1 postgres postgres 3.4K Feb 14 17:13 /hrmdb_backups/scenario3/hrmdb_data_only_20260214.sql
```

```
-rw-rw-r-- 1 postgres postgres 8.6K Feb 14 17:12 /hrmdb_backups/scenario3/hrmdb_schema_only_20260214.sql
```

```
postgres@998-986-02:~$
```

4. Create backup with both schema and data in separate files

```
echo -e "\nCreating separate schema and data files with INSERT statements..."
```

```
pg_dump -U postgres --schema-only --column-inserts hrmdb >
```

```
/hrmdb_backups/scenario3/hrmdb_schema_inserts.sql
```

```
pg_dump -U postgres --data-only --column-inserts hrmdb >/hrmdb_backups/scenario3/hrmdb_data_inserts.sql
```

```
# 5. Check first few lines of each
echo -e "\nFirst 5 lines of schema file:"
head -5 "$SCHEMA_BACKUP"
echo -e "\nFirst 5 lines of data file:"
head -5 "$DATA_BACKUP"
```

```
postgres@998-986-02:~$ echo -e "\nFirst 5 lines of schema file:"
head -5 "$SCHEMA_BACKUP"
echo -e "\nFirst 5 lines of data file:"
head -5 "$DATA_BACKUP"

First 5 lines of schema file:
--
-- PostgreSQL database dump
--

\restrict 3QIdrfRvW07S4tFGGYGOYKVTlWy9faQKQRiEXSoPl7F1y5Rw8yEdTN50XTP723y

First 5 lines of data file:
--
-- PostgreSQL database dump
--

\restrict UL4ohhFgnVrs1WIy00zVc60F03pF3o4G062UZXTJnf5cj2rEIdly70o0VgQ0RiS
postgres@998-986-02:~$
```

■ Schema and Data Restore

```
echo "=== SCENARIO 3: Schema then Data Restore ==="
```

```
# 1. Create empty database
```

```
createdb -U postgres hrmdb_schema_data_test
```

```
# 2. Restore schema first
```

```
echo "Restoring schema..."
```

```
psql -U postgres -d hrmdb_schema_data_test < "$SCHEMA_BACKUP"
```

```
postgres@998-986-02:~$ createdb -U postgres hrmdb_schema_data_test
postgres@998-986-02:~$ echo "Restoring schema..."
psql -U postgres -d hrmdb_schema_data_test < "$SCHEMA_BACKUP"
Restoring schema...
SET
SET
SET
SET
SET
SET
SET
set_config
-----

(1 row)

SET
SET
```

```
# 3. Verify schema (tables exist but empty)
```

```
echo -e "\nTables after schema restore (should be empty):"
```

```
psql -U postgres -d hrmdb_schema_data_test -c "SELECT COUNT(*) FROM employees;"
```

```
psql -U postgres -d hrmdb_schema_data_test -c "\d"
```

```
postgres@998-986-02:~$ echo -e "\nTables after schema restore (should be empty):"
psql -U postgres -d hrmdb_schema_data_test -c "SELECT COUNT(*) FROM employees;"
psql -U postgres -d hrmdb_schema_data_test -c "\d"
```

Tables after schema restore (should be empty):

```
count
-----
      0
(1 row)
```

List of relations			
Schema	Name	Type	Owner
public	audit_logs	table	postgres
public	audit_logs_log_id_seq	sequence	postgres
public	departments	table	postgres
public	departments_department_id_seq	sequence	postgres
public	employee_summary	view	postgres
public	employees	table	postgres
public	employees_employee_id_seq	sequence	postgres
public	salaries	table	postgres
public	salaries_salary_id_seq	sequence	postgres

```
postgres@998-986-02:~$
```

4. Restore data

```
echo -e "\nRestoring data..."
psql -U postgres -d hrmdb_schema_data_test < "$DATA_BACKUP"
```

```

CASE WHEN TG_OP IN ('INSERT', 'UPDATE') THEN row_to_json(NEW) ELSE NULL END
)
CONTEXT:  PL/pgSQL function public.audit_employee_changes() line 3 at SQL statement
ERROR:  insert or update on table "salaries" violates foreign key constraint "salaries_employee_id_fkey"
DETAIL:  Key (employee_id)=(1) is not present in table "employees".
setval
-----
      1
(1 row)

setval
-----
      5
(1 row)

setval
-----
     10
(1 row)

setval
-----
      5
(1 row)
```

5. Verify data

```
echo -e "\nData after restore:"
psql -U postgres -d hrmdb_schema_data_test -c "SELECT COUNT(*) FROM employees;"
psql -U postgres -d hrmdb_schema_data_test -c "SELECT * FROM departments;"
```

```

postgres@998-986-02:~$ echo -e "\nData after restore:"
psql -U postgres -d hrmdb_schema_data_test -c "SELECT COUNT(*) FROM employees;"
psql -U postgres -d hrmdb_schema_data_test -c "SELECT * FROM departments;"

Data after restore:
count
-----
      0
(1 row)

 department_id | department_name | manager_id | location | budget
-----+-----+-----+-----+-----
              1 | Engineering     |           1 | New York | 1000000.00
              2 | Human Resources |           2 | Chicago  | 500000.00
              3 | Sales           |           3 | Los Angeles | 750000.00
              4 | Marketing       |           4 | New York | 600000.00
              5 | Finance         |           5 | Chicago  | 800000.00
(5 rows)

postgres@998-986-02:~$

```

Point-in-Time Recovery (PITR)

▪ Multiple Timestamped Backups

```

mkdir -p /hrmdb_backups/scenario4

echo "=== SCENARIO 4: Point-in-Time Recovery ==="

# 1. Create initial backup
echo "Creating initial backup (State 1)..."
pg_dump -U postgres -Fc hrmdb > /hrmdb_backups/scenario4/hrmdb_state1.dump

# 2. Make some changes (simulate time passing)
echo -e "\nMaking changes to database (State 2)..."
psql -U postgres -d hrmdb << EOF
INSERT INTO employees (first_name, last_name, email, hire_date, salary, department_id)
VALUES ('Test', 'User1', 'test1@company.com', CURRENT_DATE, 50000, 1);

INSERT INTO departments (department_name, manager_id, location, budget)
VALUES ('R&D', 11, 'Boston', 2000000);

UPDATE employees SET salary = 90000 WHERE employee_id = 1;
EOF

```

```

Making changes to database (State 2)...
INSERT 0 1
INSERT 0 1
UPDATE 1
postgres@998-986-02:~$ |

```

```

# 3. Create second backup
echo "Creating second backup (State 2)..."
pg_dump -U postgres -Fc hrmdb > /hrmdb_backups/scenario4/hrmdb_state2.dump

```

```

postgres@998-986-02:~$ echo "Creating second backup (State 2)..."
pg_dump -U postgres -Fc hrmdb > /hrmdb_backups/scenario4/hrmdb_state2.dump
Creating second backup (State 2)...
postgres@998-986-02:~$

```

4. Make more changes (simulate more time passing)

```

echo -e "\nMaking more changes (State 3)..."
psql -U postgres -d hrmdb << EOF
INSERT INTO employees (first_name, last_name, email, hire_date, salary, department_id)
VALUES ('Test', 'User2', 'test2@company.com', CURRENT_DATE, 55000, 2);

DELETE FROM employees WHERE email = 'test1@company.com';
UPDATE departments SET budget = budget * 1.1;
EOF

```

```

UPDATE 1
postgres@998-986-02:~$ echo "Creating second backup (State 2)..."
pg_dump -U postgres -Fc hrmdb > /hrmdb_backups/scenario4/hrmdb_state2.dump
Creating second backup (State 2)...
postgres@998-986-02:~$ echo -e "\nMaking more changes (State 3)..."
psql -U postgres -d hrmdb << EOF
INSERT INTO employees (first_name, last_name, email, hire_date, salary, department_id)
VALUES ('Test', 'User2', 'test2@company.com', CURRENT_DATE, 55000, 2);

DELETE FROM employees WHERE email = 'test1@company.com';
UPDATE departments SET budget = budget * 1.1;
EOF

Making more changes (State 3)...
INSERT 0 1
DELETE 1
UPDATE 6
postgres@998-986-02:~$

```

5. Create third backup

```

echo "Creating third backup (State 3)..."
pg_dump -U postgres -Fc hrmdb > /hrmdb_backups/scenario4/hrmdb_state3.dump

```

```

postgres@998-986-02:~$ echo "Creating third backup (State 3)..."
pg_dump -U postgres -Fc hrmdb > /hrmdb_backups/scenario4/hrmdb_state3.dump
Creating third backup (State 3)...
postgres@998-986-02:~$ |

```

6. Show backup timeline

```

echo -e "\nBackup timeline created:"
ls -lh /hrmdb_backups/scenario4/hrmdb_state*.dump

```

```

Backup timeline created:
-rw-rw-r-- 1 postgres postgres 28K Feb 14 17:23 /hrmdb_backups/scenario4/hrmdb_state1.dump
-rw-rw-r-- 1 postgres postgres 28K Feb 14 17:24 /hrmdb_backups/scenario4/hrmdb_state2.dump
-rw-rw-r-- 1 postgres postgres 28K Feb 14 17:25 /hrmdb_backups/scenario4/hrmdb_state3.dump
postgres@998-986-02:~$

```

■ Recovering to Different Points

```
echo "=== SCENARIO 4: Recovering to Different Points ==="
```

```
# 1. Restore to State 1 (initial state)
```

```
echo "Restoring to State 1 (initial)..."
```

```
createdb -U postgres hrmdb_state1_recovered
```

```
pg_restore -U postgres -d hrmdb_state1_recovered /hrmdb_backups/scenario4/hrmdb_state1.dump
```

```
# 2. Check data in State 1
```

```
echo -e "\nData in State 1 (should have original employees):"
```

```
psql -U postgres -d hrmdb_state1_recovered -c "SELECT COUNT(*) FROM employees;"
```

```
psql -U postgres -d hrmdb_state1_recovered -c "SELECT first_name, last_name FROM employees ORDER BY employee_id LIMIT 5;"
```

```
postgres@998-986-02:~$ echo "Restoring to State 1 (initial)..."
createdb -U postgres hrmdb_state1_recovered
pg_restore -U postgres -d hrmdb_state1_recovered /hrmdb_backups/scenario4/hrmdb_state1.dump
Restoring to State 1 (initial)...
postgres@998-986-02:~$ echo -e "\nData in State 1 (should have original employees):"
psql -U postgres -d hrmdb_state1_recovered -c "SELECT COUNT(*) FROM employees;"
psql -U postgres -d hrmdb_state1_recovered -c "SELECT first_name, last_name FROM employees ORDER BY employee_id LIMIT 5;"
```

```
Data in State 1 (should have original employees):
```

```
count
```

```
-----
```

```
10
```

```
(1 row)
```

```
first_name | last_name
```

```
-----+-----
```

```
John
```

```
Jane
```

```
Bob
```

```
Alice
```

```
Charlie
```

```
(5 rows)
```

```
# 3. Restore to State 2 (after first changes)
```

```
echo -e "\nRestoring to State 2 (after first changes)..."
```

```
createdb -U postgres hrmdb_state2_recovered
```

```
pg_restore -U postgres -d hrmdb_state2_recovered /hrmdb_backups/scenario4/hrmdb_state2.dump
```

```
# 4. Check State 2 data
```

```
echo -e "\nData in State 2 (should include Test User1):"
```

```
psql -U postgres -d hrmdb_state2_recovered -c "SELECT first_name, last_name, email FROM employees WHERE email LIKE 'test%';"
```

```
postgres@998-986-02:~$ echo -e "\nRestoring to State 2 (after first changes)..."
```

```
createdb -U postgres hrmdb_state2_recovered
```

```
pg_restore -U postgres -d hrmdb_state2_recovered /hrmdb_backups/scenario4/hrmdb_state2.dump
```

```
Restoring to State 2 (after first changes)...
```

```
postgres@998-986-02:~$ echo -e "\nData in State 2 (should include Test User1):"
```

```
psql -U postgres -d hrmdb_state2_recovered -c "SELECT first_name, last_name, email FROM employees WHERE email LIKE 'test%';"
```

```
Data in State 2 (should include Test User1):
```

```
first_name | last_name | email
```

```
-----+-----+-----
```

```
Test
```

```
(1 row)
```

```
# 5. Restore to State 3 (final state)
echo -e "\nRestoring to State 3 (final)..."
createdb -U postgres hrmdb_state3_recovered
pg_restore -U postgres -d hrmdb_state3_recovered /hrmdb_backups/scenario4/hrmdb_state3.dump
```

```
# 6. Check State 3 data
echo -e "\nData in State 3 (should have Test User2 only):"
psql -U postgres -d hrmdb_state3_recovered -c "SELECT first_name, last_name, email FROM employees WHERE email LIKE 'test%';"
```

```
postgres@998-986-02:~$ echo -e "\nRestoring to State 3 (final)..."
createdb -U postgres hrmdb_state3_recovered
pg_restore -U postgres -d hrmdb_state3_recovered /hrmdb_backups/scenario4/hrmdb_state3.dump

Restoring to State 3 (final)...
postgres@998-986-02:~$ echo -e "\nData in State 3 (should have Test User2 only):"
psql -U postgres -d hrmdb_state3_recovered -c "SELECT first_name, last_name, email FROM employees WHERE email LIKE 'test%';"

Data in State 3 (should have Test User2 only):
 first_name | last_name | email
-----+-----+-----
Test       | User2     | test2@company.com
(1 row)

postgres@998-986-02:~$
```

```
# 7. Compare employee counts across states
echo -e "\nEmployee counts across different recovery points:"
psql -U postgres -d hrmdb_state1_recovered -c "SELECT 'State 1' as recovery_point, COUNT(*) as employees FROM employees;"
psql -U postgres -d hrmdb_state2_recovered -c "SELECT 'State 2' as recovery_point, COUNT(*) as employees FROM employees;"
psql -U postgres -d hrmdb_state3_recovered -c "SELECT 'State 3' as recovery_point, COUNT(*) as employees FROM employees;"
```

```
postgres@998-986-02:~$ echo -e "\nEmployee counts across different recovery points:"
psql -U postgres -d hrmdb_state1_recovered -c "SELECT 'State 1' as recovery_point, COUNT(*) as employees FROM employees;"
psql -U postgres -d hrmdb_state2_recovered -c "SELECT 'State 2' as recovery_point, COUNT(*) as employees FROM employees;"
psql -U postgres -d hrmdb_state3_recovered -c "SELECT 'State 3' as recovery_point, COUNT(*) as employees FROM employees;"

Employee counts across different recovery points:
 recovery_point | employees
-----+-----
State 1        |         10
(1 row)

 recovery_point | employees
-----+-----
State 2        |         11
(1 row)

 recovery_point | employees
-----+-----
State 3        |         11
(1 row)

postgres@998-986-02:~$ |
```

Advanced Backup and Restore with Database Objects.

```
mkdir -p /hrmdb_backups/scenario5

echo "=== SCENARIO 5: Advanced Backup with Special Objects ==="
```

1. Backup with all objects and privileges

```
ADVANCED_BACKUP="/hrmdb_backups/scenario5/hrmdb_advanced_$(date +%Y%m%d).dump"
echo "Creating advanced backup with all objects and privileges..."
pg_dump -U postgres -Fc \
  --blobs \
  --encoding=UTF8 \
  hrmdb > "$ADVANCED_BACKUP"
```

```
postgres@998-986-02:~$ ADVANCED_BACKUP="/hrmdb_backups/scenario5/hrmdb_advanced_$(date +%Y%m%d).dump"
echo "Creating advanced backup with all objects and privileges..."
pg_dump -U postgres -Fc \
  --blobs \
  --encoding=UTF8 \
  hrmdb > "$ADVANCED_BACKUP"
Creating advanced backup with all objects and privileges...
postgres@998-986-02:~$
```

2. Backup with compression level 9 (maximum)

```
COMPRESSED_BACKUP="/hrmdb_backups/scenario5/hrmdb_max_compressed.dump"
echo -e "\nCreating maximum compressed backup..."
pg_dump -U postgres -Fc -Z 9 hrmdb > "$COMPRESSED_BACKUP"
```

3. Create a backup with INSERT commands for compatibility

```
INSERT_BACKUP="/hrmdb_backups/scenario5/hrmdb_insert_format.sql"
echo -e "\nCreating backup with INSERT statements (for cross-version compatibility)..."
pg_dump -U postgres --column-inserts --rows-per-insert=10 hrmdb > "$INSERT_BACKUP"
```

4. Compare backup sizes

```
echo -e "\nBackup size comparison:"
ls -lh "$ADVANCED_BACKUP" "$COMPRESSED_BACKUP" "$INSERT_BACKUP" | awk '{print $5 "\t" $9}'
```

```
postgres@998-986-02:~$ echo -e "\nBackup size comparison:"
ls -lh "$ADVANCED_BACKUP" "$COMPRESSED_BACKUP" "$INSERT_BACKUP" | awk '{print $5 "\t" $9}'

Backup size comparison:
28K    /hrmdb_backups/scenario5/hrmdb_advanced_20260214.dump
13K    /hrmdb_backups/scenario5/hrmdb_insert_format.sql
28K    /hrmdb_backups/scenario5/hrmdb_max_compressed.dump
postgres@998-986-02:~$
```

▪ Advanced Restore with Options

```
echo "=== SCENARIO 5: Advanced Restore with Options ==="
```

1. Create database for advanced restore

```
createdb -U postgres hrmdb_advanced_restore
```

2. Restore with parallel jobs

```
echo "Restoring with parallel jobs (using 4 cores)..."
time pg_restore -U postgres -d hrmdb_advanced_restore \
  -j 4 \
  --verbose \
  "$ADVANCED_BACKUP"
```

```

postgres@998-986-02:~$ echo "Restoring with parallel jobs (using 4 cores)..."
time pg_restore -U postgres -d hrmdb_advanced_restore \
    -j 4 \
    --verbose \
    "$ADVANCED_BACKUP"
Restoring with parallel jobs (using 4 cores)...
pg_restore: connecting to database for restore
pg_restore: executing SELECT pg_catalog.set_config('search_path', '', false);
pg_restore: processing item 3468 ENCODING ENCODING
pg_restore: processing item 3469 STDSTRINGS STDSTRINGS
pg_restore: processing item 3470 SEARCHPATH SEARCHPATH
pg_restore: processing item 3471 DATABASE hrmdb
pg_restore: processing item 228 FUNCTION audit_employee_changes()
pg_restore: creating FUNCTION "public.audit_employee_changes()"
pg_restore: processing item 229 FUNCTION get_employees_by_department(integer)
pg_restore: creating FUNCTION "public.get_employees_by_department(integer)"
pg_restore: processing item 226 TABLE audit_logs

```

3. Check if all objects were restored

```

echo -e "\nVerifying special objects:"
echo "Functions:"
psql -U postgres -d hrmdb_advanced_restore -c "\df"
echo -e "\nViews:"
psql -U postgres -d hrmdb_advanced_restore -c "\dv"
echo -e "\nTriggers:"
psql -U postgres -d hrmdb_advanced_restore -c "SELECT trigger_name, event_manipulation FROM
information_schema.triggers WHERE trigger_schema = 'public';"

```

```

postgres@998-986-02:~$ psql -U postgres -d hrmdb_advanced_restore -c "\dv"
          List of views
 Schema |      Name      | Type | Owner
-----+-----+-----+-----
 public | employee_summary | view | postgres
(1 row)

postgres@998-986-02:~$ psql -U postgres -d hrmdb_advanced_restore -c "SELECT trigger_name, event_manipulation FROM information_schema
.triggers WHERE trigger_schema = 'public';"
      trigger_name      | event_manipulation
-----+-----
 employee_audit_trigger | INSERT
 employee_audit_trigger | DELETE
 employee_audit_trigger | UPDATE
(3 rows)

postgres@998-986-02:~$
          List of functions
 Schema |      Name      |      Result data type      | Argumen
-----+-----+-----+-----
 public | audit_employee_changes | trigger |
 public | get_employees_by_department | TABLE(employee_id integer, full_name text, email character varying, salary numeric) | dept_id integer
(2 rows)

~
~
~

```